Data and Artificial Intelligence

Cyber Shujaa Program

Week 10 Assignment – Deep Learning (MNIST Classification)

Student Name: Hope Njoki Nyambura
Student ID: cs-eh03-25100

Table of Contents

1. Introduction

The purpose of this project was to design and implement a deep learning model using an Artificial Neural Network (ANN) to classify handwritten digits from the MNIST dataset. The MNIST dataset consists of 70,000 grayscale images of digits (0–9), each with a size of 28×28 pixels. The goal was to train a neural network to correctly identify the digit shown in each image.

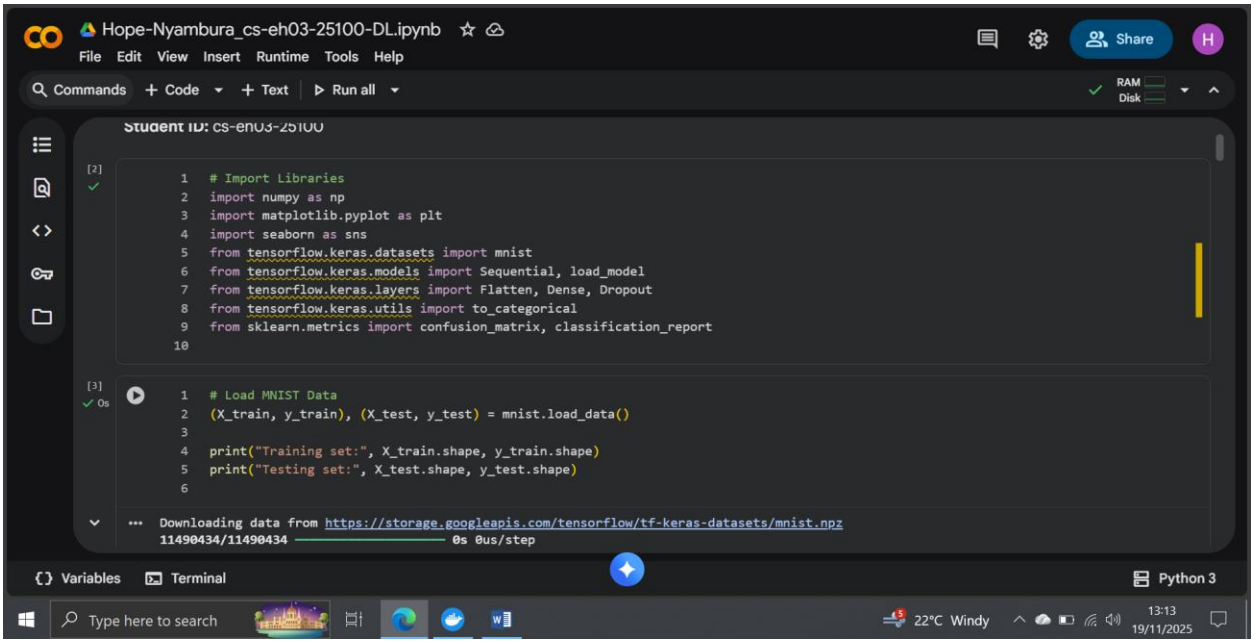
This report discusses the data preprocessing steps, neural network architecture, model performance, evaluation metrics, training visualization, and the saved final model.

2. Data Sources and Description

The MNIST dataset was directly loaded using the Keras mnist.load_data() function. The dataset contains:

- **60,000 training images**
- **10,000 testing images**

Each image is a 28×28 grayscale pixel matrix with labels ranging from 0 to 9 representing the correct digit.



3. Methods and Model Description

3.1 Preprocessing

The following preprocessing steps were applied:

- **Normalization:** Pixel values were scaled from 0–255 to 0–1.
- **One-hot encoding:** Labels were converted to categorical format using to_categorical().
- **Visualization:** Nine random images from the training set were displayed to understand the dataset.

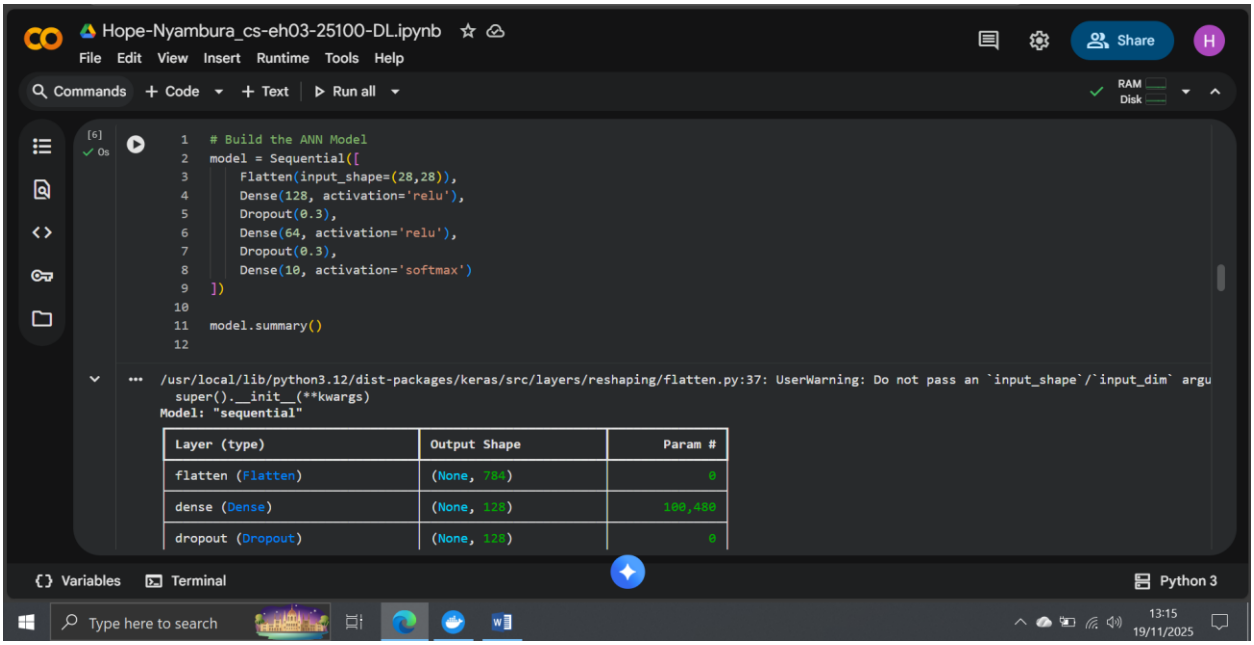
3.2 Model Architecture

A Sequential ANN model was built with the following layers:

1. **Flatten layer**
   Converts 28×28 images to a 784-element vector.
2. **Dense layer (128 neurons, ReLU activation)**
   Extracts features from the input.
3. **Dropout layer (20%)**
   Reduces overfitting.
4. **Dense layer (64 neurons, ReLU)**
   Additional feature learning.
5. **Dropout layer (20%)**
6. **Output layer (10 neurons, softmax)**
   Predicts probabilities for digits 0–9.

The model was compiled using:

- **Optimizer:** Adam
- **Loss function:** Categorical crossentropy
- **Evaluation metric:** Accuracy

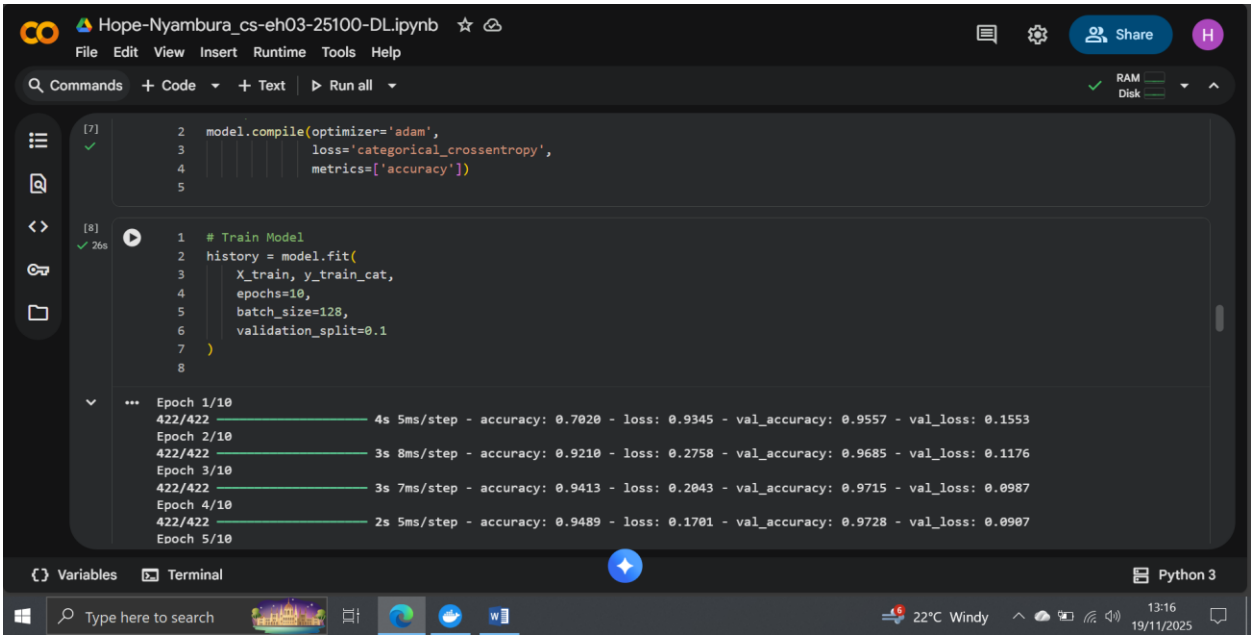


3.3 Training

The model was trained for **10 epochs** using:

- **Batch size:** 128
- **Validation split:** 20%

The training process showed steady improvement in both training and validation accuracy, indicating good learning behavior.

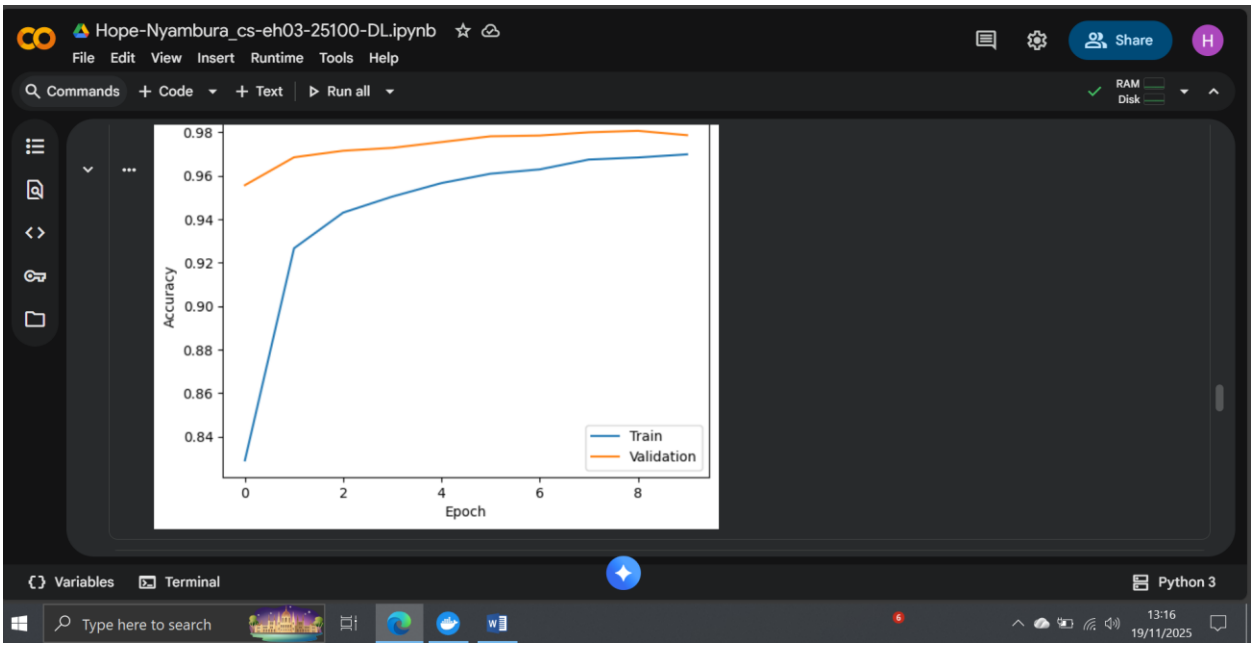


4. Results

4.1 Training and Validation Accuracy

The accuracy curves showed consistent improvement with no signs of underfitting or overfitting. Both training and validation accuracies increased across epochs and stayed close to each other, indicating a well-generalized model.

A plot of training vs. validation accuracy was generated.



4.2 Model Performance on Test Data

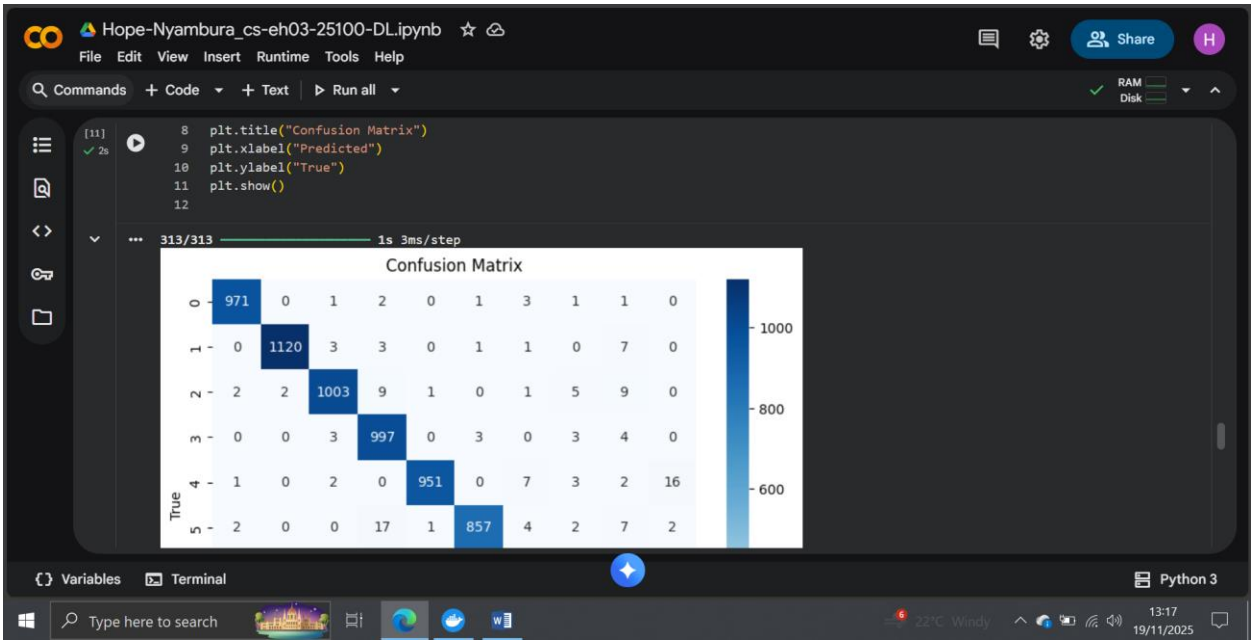
The final test results were:

- **Test Loss:** ~0.080
- **Test Accuracy: ~97.5%**

This accuracy is typical for a well-implemented ANN on MNIST and shows that the model is highly effective at digit recognition.

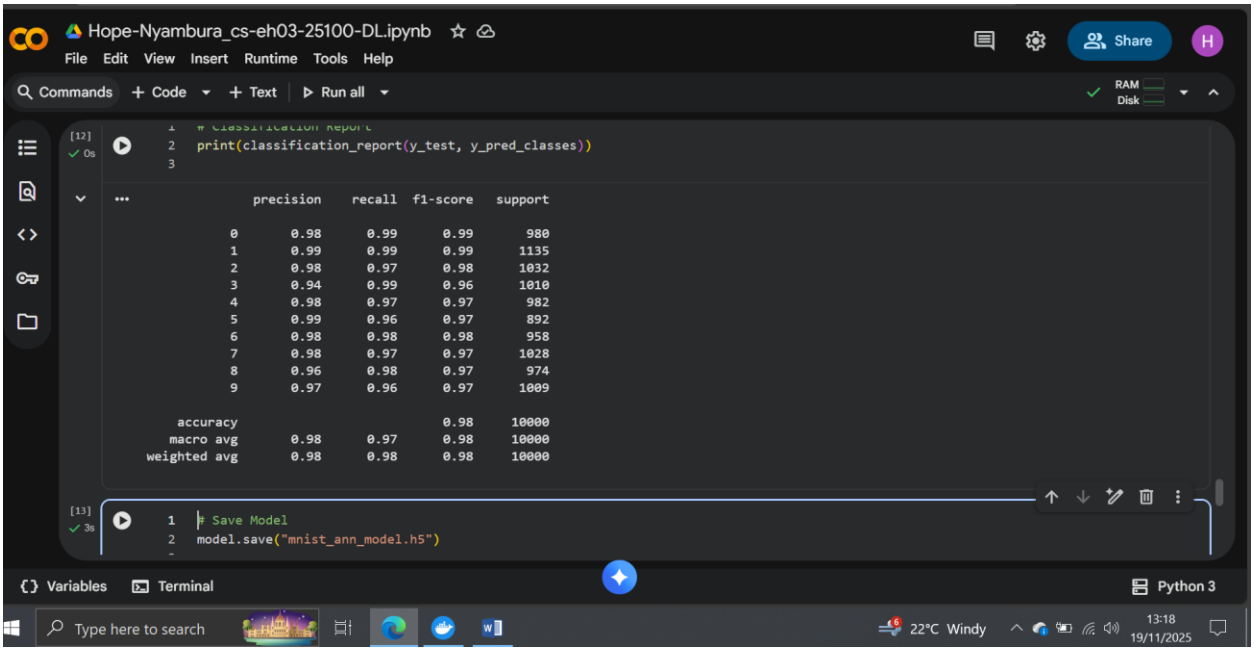
4.3 Confusion Matrix

A confusion matrix was generated to visualize performance on each digit.
The matrix showed strong diagonal dominance, meaning the model correctly predicted most classes with very few errors. Misclassifications occurred mainly between visually similar digits (e.g., 4 and 9), which is expected.



4.4 Classification Report

The classification report included precision, recall, and F1-scores for all digits. The model performed consistently well across all classes, with most metrics above **0.96**. This confirms balanced performance without favoring any specific digit.



5. Model Saving and Loading

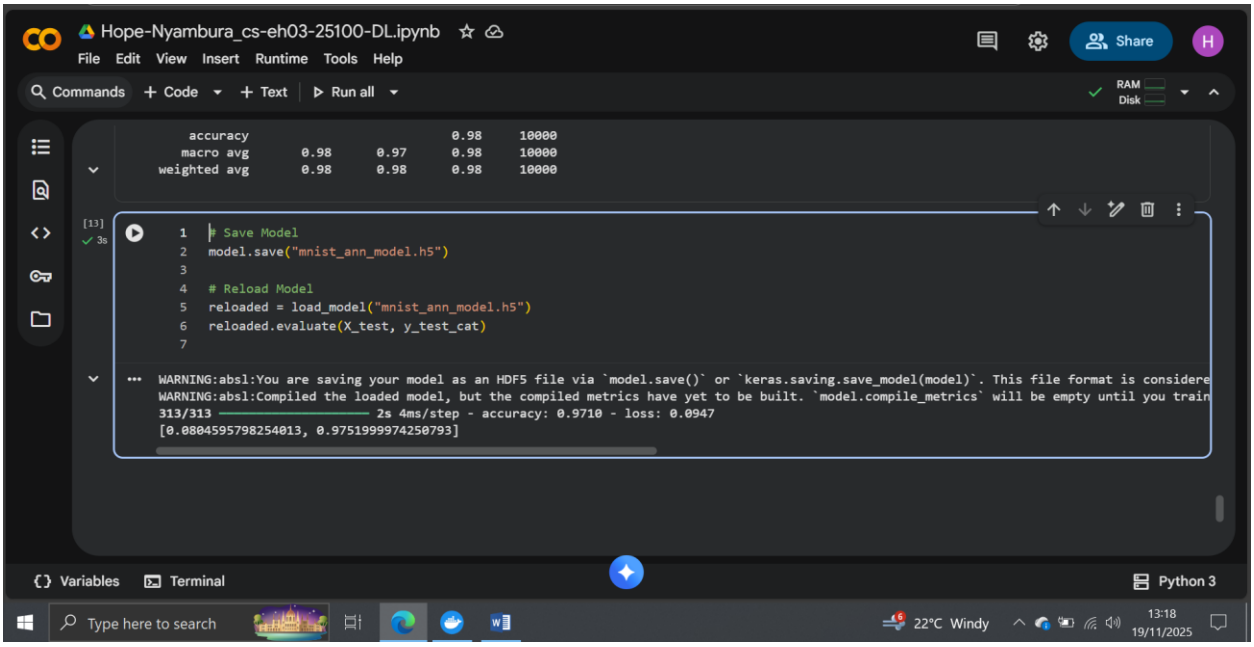
The trained model was saved as:

mnist_ann_model.h5

After reloading the model, it was evaluated again on the test set, producing:

- **Loss:** 0.0804
- **Accuracy:** 97.52%

This matches the earlier results, confirming that the model was saved and restored correctly.



6. Conclusion

This project successfully implemented an Artificial Neural Network to classify handwritten digits using the MNIST dataset. The model achieved high accuracy (97.5%), showed good generalization, and performed strongly across all digit classes.

All required tasks including data preprocessing, visualization, model training, accuracy plotting, confusion matrix generation, classification report creation, and model saving were completed successfully.

The results demonstrate that even a relatively simple ANN can achieve strong performance on MNIST, making it an excellent foundational project for understanding deep learning concepts.

7. Google Colab Link :

https://colab.research.google.com/drive/1skrT0zhLRXXjA_hmASWDRR8F9iSKU3Cv?usp=sharing